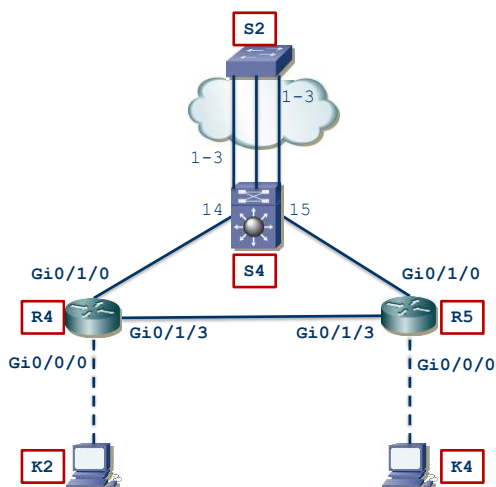


Topologia na dziś



- Role urządzeń:
 - S2: nic nie robi
 - S4: agreguje sieci, realizuje politykę routingu
 - R4, R5: urządzenia dostępne + routing OSPF
- Konstruujemy cztery sieci:
 - usługi wspólne (RD 65500:0)
 - AGH (RD 65500:1)
 - PKR (RD 65500:2)
 - UJAG (RD 65500:3)

2

2

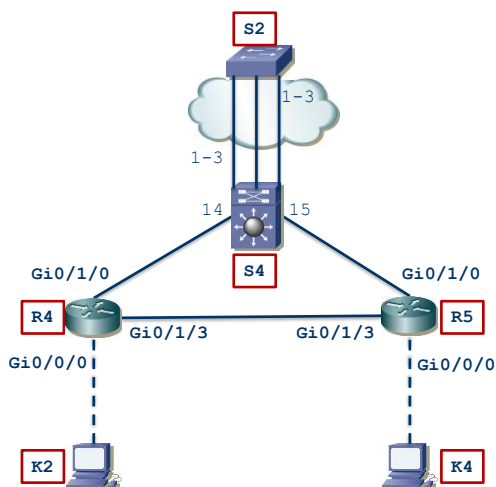
Krok -1: Przygotowanie PC

- Podłączamy wszystkie PC do Internetu
- Ściągamy biblioteki netmiko, time i getpass

3

3

Krok 0: budujemy topologię fizyczną



- Podłączamy konsole
- Łączymy kablami
- Włączamy do prądu...

4

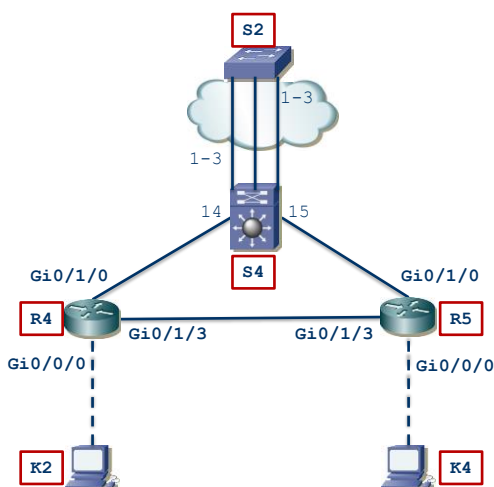
4

Krok 1: wstępna konfiguracja urządzeń

- Na R4, R5 i S5 wykonujemy:

```
enable
config t
line console 0
  logging synchronous
  exec-timeout 0 0
exit
no ip domain lookup
```

```
hostname {R4|R5|S5}
ip routing
vtp mode transparent
```



5

5

Krok 2: Konfiguracja VRF

- Przykład: zakładamy VRF AGH, nadajemy jej identyfikator

```
enable
config t
  ip vrf AGH
  rd 65500:1
exit
```

- Samodzielnie (!) zakładamy następujące VRF:

Nazwa	RD	R4	R5	S4
Common	65500:0			+
AGH	65500:1	+	+	+
PKR	65500:2	+		+
UJAG	65500:3		+	+

- Nie usuwamy i nie modyfikujemy VRF Internal(!)

6

6

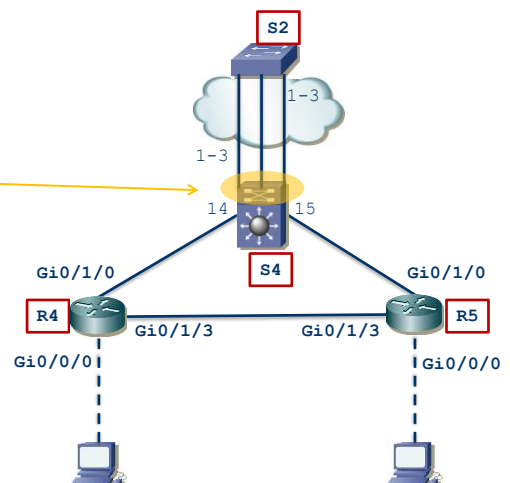
Krok 3: dodajemy interfejsy do VRF

- Przykład: dodanie interfejsu fizycznego do VRF

```
enable
config t
  int gi1/0/1
  description AGH-to-Internet
  no switchport
  ip vrf forwarding AGH
  ip address 172.16.1.1 255.255.255.0
exit
```

- Konfigurujemy interfejsy 1, 2 i 3 na S4

- Gi1/0/1: VRF AGH, 172.16.1.1/24
- Gi1/0/2: VRF PKR, 172.16.2.1/24
- Gi1/0/3: VRF UJAG, 172.16.3.1/24



7

7

Krok 3: dodajemy nowe interfejsy do VRF

- Przykład: dodanie interfejsu loopback do VRF

```
enable
config t
  int loopback 1
    description AGH-priv-service
    ip vrf forwarding AGH
    ip address 10.0.1.1 255.255.255.255
  exit
```

- Konfigurujemy interfejsy loopback 0, 1, 2 i 3 na S4
 - lo0: VRF Common, 10.0.0.1/32
 - lo1: VRF AGH, 10.0.1.1/32
 - lo2: VRF PKR, 10.0.2.1/32
 - lo3: VRF UJAG, 10.0.3.1/32

8

8

Krok 3: dodajemy interfejsy do VRF

- Przykład: dodanie interfejsu loopback do VRF

```
enable
config t
  int loopback 1
    description AGH-priv-service
    ip vrf forwarding AGH
    ip address 10.0.1.1 255.255.255.255
  exit
```

- Analogicznie konfigurujemy interfejsy 1 i 2 na R4...
 - lo1: VRF AGH, 10.0.1.2/32
 - lo2: VRF PKR, 10.0.2.2/32
- ... oraz 1 i 3 na R5
 - lo1: VRF AGH, 10.0.1.3/32
 - lo3: VRF UJAG, 10.0.3.3/32

9

9

Krok 3.1: weryfikacja

S4#show ip vrf interfaces | exclude unassigned

Interface	IP-Address	VRF	Protocol
Gi1/0/1	172.16.1.1	AGH	up
Lo1	10.0.1.1	AGH	up
Lo0	10.0.0.1	Common	up
Gi1/0/2	172.16.2.1	PKR	up
Lo2	10.0.2.1	PKR	up
Gi1/0/3	172.16.3.1	UJAG	up
Lo3	10.0.3.1	UJAG	up

Mogą również istnieć inne wpisy,
nie utworzone przez nas

R4#show ip vrf interfaces | exclude unassigned

Interface	IP-Address	VRF	Protocol
Lo1	10.0.1.2	AGH	up
Lo2	10.0.2.2	PKR	up

R5#show ip vrf interfaces | exclude unassigned

Interface	IP-Address	VRF	Protocol
Lo1	10.0.1.3	AGH	up
Lo3	10.0.3.3	UJAG	up

10

10

Krok 4: definiujemy VLANy, trunki itp.

- Definiujemy trunki między S4, R4 i R5:

S4# enable
config t

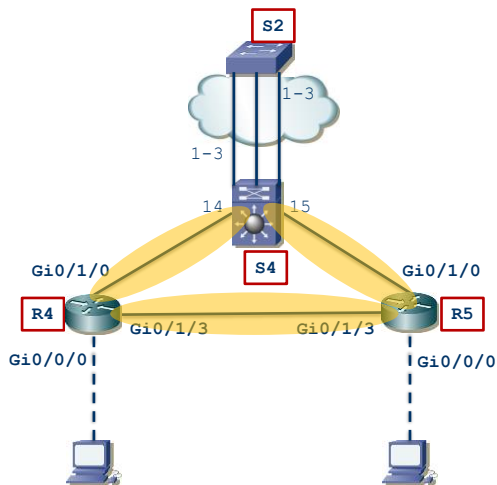
```
interface range gi1/0/14 - 15
switchport mode trunk
exit
```

R4# enable
config t

```
interface range gi0/1/0 , gi0/1/3
switchport mode trunk
exit
```

R5# enable
config t

```
interface range gi0/1/0 , gi0/1/3
switchport mode trunk
exit
```



11

11

Krok 4: definiujemy VLANy, trunki itp.

- Przykład: zdefiniowanie VLAN 101, konfiguracja SVI

```
enable
config t
  vlan 101
    name AGH-transit
  exit
  interface vlan 101
    description AGH-transit
    ip vrf forwarding AGH
    ip address 10.1.1.1 255.255.255.0
  exit
```

- Na S4 konfigurujemy:
 - VLAN 101 (AGH-transit), 10.1.1.1/24, VRF AGH
 - VLAN 102 (PKR-transit), 10.1.2.1/24, VRF PKR
 - VLAN 103 (UJAG-transit), 10.1.3.1/24, VRF UJAG

12

12

Krok 4: definiujemy VLANy, trunki itp.

- ... analogicznie definiujemy:
 - na R4:
 - VLAN 101 (AGH-transit), 10.1.1.2/24, VRF AGH
 - VLAN 102 (PKR-transit), 10.1.2.2/24, VRF PKR
 - VLAN 201 (AGH-hosts), 10.2.1.2/24, VRF AGH
 - VLAN 202 (PKR-hosts), 10.2.2.2/24, VRF PKR
 - na R5:
 - VLAN 101 (AGH-transit), 10.1.1.3/24, VRF AGH
 - VLAN 103 (UJAG-transit), 10.1.3.3/24, VRF UJAG
 - VLAN 301 (AGH-hosts), 10.3.1.3/24, VRF AGH
 - VLAN 303 (UJAG-hosts), 10.3.3.3/24, VRF UJAG
- Do VLANów *-hosts można dopiąć PC z bramą jak na SVI

13

13

Technologie internetu przyszłości

Laboratorium 2: MP-BGP w sieci z VRF

rysunki oparte na materiałach Cisco Systems Networking Academy

© Sławomir Zieliński, KT AGH 2025

Krok 5: OSPF

- Przykład konfiguracji dla S4, VRF AGH:

```

enable
config t
  router ospf 1 vrf AGH
  capability vrf-lite
  network 10.0.0.0 0.255.255.255 area 100
exit

```
- Konfigurujemy:
 - na R4, R5 i S4: OSPF 1 dla VRF AGH
 - na R4 i S4: OSPF 2 dla VRF PKR
 - na R5 i S4: OSPF 3 dla VRF UJAG

14

Technologie internetu przyszłości

Laboratorium 2: MP-BGP w sieci z VRF

rysunki oparte na materiałach Cisco Systems Networking Academy

© Sławomir Zieliński, KT AGH 2025

Krok 5.1: Weryfikacja

S4#show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
10.0.3.3	1	FULL/BDR	00:00:39	10.1.3.3	Vlan103
10.0.2.2	1	FULL/DR	00:00:38	10.1.2.2	Vlan102
10.0.1.2	1	FULL/BDR	00:00:36	10.1.1.2	Vlan101
10.0.1.3	1	FULL/DR	00:00:39	10.1.1.3	Vlan101

R4#show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
10.0.2.1	1	FULL/BDR	00:00:35	10.1.2.1	Vlan102
10.0.1.1	1	FULL/DROTHER	00:00:35	10.1.1.1	Vlan101
10.0.1.3	1	FULL/DR	00:00:30	10.1.1.3	Vlan101

R5#show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
10.0.3.1	1	FULL/DR	00:00:31	10.1.3.1	Vlan103
10.0.1.1	1	FULL/DROTHER	00:00:31	10.1.1.1	Vlan101
10.0.1.2	1	FULL/BDR	00:00:35	10.1.1.2	Vlan101

Role DR/BDR mogą być przypisane innym routerom – to zależy m.in. od kolejności startowania procesów

15

Krok 5.1: Weryfikacja

```
R4#show ip route vrf PKR
[...]
10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
O      10.0.2.1/32 [110/2] via 10.1.2.1, 00:08:11, Vlan102
C      10.0.2.2/32 is directly connected, Loopback2
C      10.1.2.0/24 is directly connected, Vlan102
L      10.1.2.2/32 is directly connected, Vlan102
C      10.2.2.0/24 is directly connected, Vlan202
L      10.2.2.2/32 is directly connected, Vlan202
```

Inne – podobnie...

Wniosek: separacja na warstwie 3 jest zachowana.

UWAGA: wpisy „L” pomijamy – są nieistotne z naszego punktu widzenia.

16

16

Krok 6: MP-BGP

- Na R4 i R5 włączamy śledzenie...

```
enable
debug ip routing
```

- ... a na S4 konfigurujemy

```
enable
config t
```

```
router bgp 65500
bgp router-id 1.1.1.1
bgp log-neighbor-changes
no auto-summary
```

```
address-family ipv4 vrf Common
redistribute connected
no synchronization
exit-address-family
```

```
address-family ipv4 vrf AGH
redistribute connected
redistribute ospf 1
no synchronization
exit-address-family
```

VRF PKR i UJAG - analogicznie

[...]

17

17

Krok 7: route leaking

- Na razie śledzenie na routerach nic nie wykazało
- Route leaking to bajecznie prosta technika...
 - ... pod warunkiem, że wcześniej przygotowujemy maszynię, ale to już zrobiliśmy
- Na S4 konfigurujemy przekazywanie tras pomiędzy VRF Common, a AGH:

```
router ospf 1
 redistribute bgp 65500 subnets
```

OSPF 2 i 3: analogicznie...

```
ip vrf AGH
 route-target export 65500:1
 route-target import 65500:0
```

```
ip vrf Common
 route-target export 65500:0
 route-target import 65500:1
```

Common ↔ PKR: analogicznie...

Common ↔ UJAG: analogicznie...

18

18

Krok 7.1: Quod erat demonstrandum

- Nawet BGP uruchomiony na jednym switchu działa (!)

```
R4#
*Mar  7 00:55:04.585: RT(AGH): updating ospf 10.0.0.1/32 (0x2)  :
    via 10.1.1.1 V1101  0 1048578

*Mar  7 00:55:04.585: RT(AGH): add 10.0.0.1/32 via 10.1.1.1, ospf metric [110/1]
*Mar  7 00:55:04.586: RT(PKR): updating ospf 10.0.0.1/32 (0x3)  :
    via 10.1.2.1 V1102  0 1048578

*Mar  7 00:55:04.586: RT(PKR): add 10.0.0.1/32 via 10.1.2.1, ospf metric [110/1]

R4#show ip route vrf AGH
[...]
```

O E2 10.0.0.1/32 [110/1] via 10.1.1.1, 00:01:49, Vlan101

[...]

Przeciekają tylko te trasy, którym kazaliśmy przeciekać (!)

19

19

Krok 8: A co z hostami?

- Interfejsy gi0/0/0 routerów konfigurujemy tak, by należały do jednej z sieci klienckich (niekoniecznie tej samej)
 - przypisanie do VRF,
 - adresacja,
 - ogłoszenie w OSPF
- Konfigurujemy zdalny dostęp (przez SSH) na routerach

```
conf t
hostname R4
ip domain name tip.agh
crypto key zeroize rsa
crypto key generate rsa modulus 2048 general-keys
```

```
line vty 0 4
transport input ssh
login local
enable secret inf3
username inf3 privil 15 password inf3
```

Może nie działać – testowałem bez VRFów,
a potem przepisywałem z pamięci :-\.

20

20

Krok 8.1: Python, skrypt i takie tam...

- Wykorzystujemy bibliotekę netmiko do:
 - pobrania listy interfejsów routera
 - pobrania listy opisów interfejsów routera
 - utworzenia i ogłoszenia w OSPF dla VRF UJAG dwudziestu nowych sieci
 - i jeszcze dwudziestu...
 - ...
 - ...
 - ... może już wystarczy ...
- Obserwujemy efekty w tablicach routingu
- Usuwamy sieci wytworzone skryptem

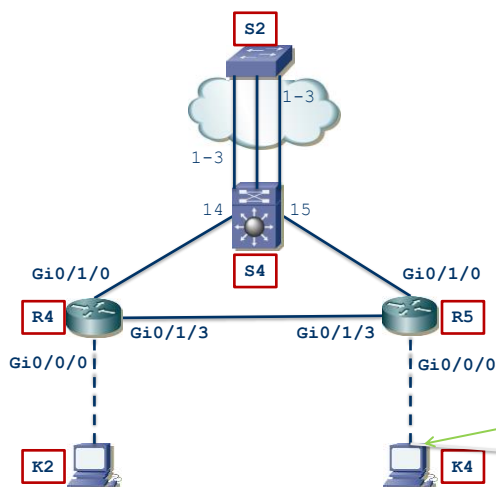
Wskazówka: można posłużyć się zawartością pliku tip-lab02-*.py, ale proszę o własne skrypty(!)
ConnectHandler – „trzymaacz” sesji, trzeba ten obiekt utworzyć na początku konwersacji

Może nie działać – testowałem bez VRFów,
a potem przepisywałem z pamięci :-\.

21

21

Opcja – router wirtualny



- Na K4 uruchamiamy VirtualBox
 - importujemy [Ctrl-I] TC-Router-11
 - pierwszą kartę sieciową [Network->Settings] ustawiamy w tryb mostkowany do Ethernetu
 - drugą – w tryb mostkowany do sieci wewnętrznej „intnet-11”
 - importujemy tinyCore-11 i tinyCore-12
 - ich karty sieciowe ustawiamy w tryb mostkowany do „intnet-11”
 - dodajemy serwis DHCP
 - startujemy maszyny
 - ustawiamy adres IP na eth0 routera tcR11

• Obserwujemy tablice routingu

```
C:\Program Files\Oracle\VirtualBox (jedna linia):
vboxmanage dhcpserver add --netname intnet-11
--ip 10.10.103.1 --netmask 255.255.255.0
--lowerip 10.10.103.11 --upperip 10.10.103.254 --enable
```



22

22

I tyle...

- ... ostatni wyłączają zasilanie szaf

23

23